Coffer

Glamsterdam finding in the listing payout

Coffer Marketplace

Prepared by the Coffer team · 26 October 2026

> **Found and fixed by the Coffer team after the Zealynx audit. This finding and its fix are not part of that audit or its report.**
>
> The Zealynx audit covered this marketplace at commit 9262181, and its mitigation review covered the fixes up to commit 9383984. The report was added in commit 3aa9c61, and everything after that commit is outside the audit. The fix described here is in the commit that adds this document.

Summary

- **Finding.** Under the Glamsterdam gas rules, the buyer's transaction gas limit could decide whether a listing's seller was paid in ETH or in WETH. The marketplace was designed and tested so that it never could.
- **Severity.** Low, by our own rating. No value is lost, because the seller receives WETH one for one.
- **Fix.** A listing fill now pays the seller in ETH or reverts as a whole. The fixed gas budget and the WETH fallback are gone.
- **Exposure.** Glamsterdam is not active on mainnet, and the marketplace changed before its first mainnet deployment.

How we found it

ChainSecurity's article on Glamsterdam gas, published on 13 October 2026, shows how the new gas rules can let a failure path complete where it used to revert. That weakens contracts that catch a failed call and carry on, and the article advises letting a failed call revert the whole operation where possible. Our listing payout caught a failed ETH payment and paid the seller in WETH instead, which is that pattern. We checked it against the new rules on our own contracts, found the case where it breaks, compared three fixes and tested the one we chose.

What Glamsterdam changes

Two EIPs scheduled for Glamsterdam change what the payout costs. Both are in Last Call until 1 November 2026, so their numbers can still change.

- **EIP-8037** charges for new state by the byte, at 1,530 gas per byte. Filling an empty storage slot creates 64 bytes of new state, so it costs 64 × 1,530 = 97,920 gas on top of the write itself. A transaction can use at most 16,777,216 gas for normal work (EIP-7825), and any gas above that

goes into a separate reservoir. The reservoir pays for new state first, and every call passes it on in full, whatever gas limit the call itself sets.

- **EIP-8038** raises the cost of writes and account access. Changing a storage slot now costs 10,000 gas, plus 2,100 for the first touch of that slot in a transaction. For a slot that already holds a value, that is 12,100 where today it is 5,000. A call that sends ETH goes from 9,000 to 11,300 gas.

So one new storage slot, written for the first time in a transaction, costs this much gas.

Rules	Touch the slot	Write	New state	Total
Current (Osaka)	2,100	20,000	included in the write	22,100
Glamsterdam, no reservoir	2,100	10,000	97,920	110,020
Glamsterdam, the reservoir pays the new state	2,100	10,000	paid by the reservoir	12,100

The finding

At commit 9383984, the last commit the audit covered, a listing fill paid the seller like this.

```
// The forwarded gas is bounded, so a seller whose receive burns gas cannot leave the
// WETH fallback that follows without enough gas to run.
// slither-disable-next-line arbitrary-send-eth,calls-loop
bool okSeller = _safeTransferETH(_seller, _price, SELLER_PAYOUT_GAS_LIMIT);
if (!okSeller) {
    IWETH(I_WETH).deposit{value: _price}();
    IERC20(I_WETH).safeTransfer(_seller, _price);
}
```

When a buyer fills a listing, the marketplace sends the price to the seller's wallet and lets the wallet use at most 100,000 gas (`SELLER_PAYOUT_GAS_LIMIT`). The EVM adds a stipend of 2,300 gas to every call that sends ETH, so the wallet can use at most 102,300 gas in total. When its `receive` needs more, the call fails, and the marketplace wraps the price into WETH and sends that instead. A fill never failed on the payout.

Our tests pinned that the buyer's gas limit never picks the asset, and under the current rules that holds. A buyer cannot push a seller into WETH by sending less gas, because a call can pass on at most 63/64 of the gas that is left. When the buyer sends too little for the seller's 102,300, the marketplace keeps at most about 1,600 gas after the seller's call. The WETH fallback needs more than that just to start, 11,600 gas today and 14,300 under Glamsterdam, so the whole fill reverts instead of settling in WETH.

Under Glamsterdam it no longer holds. Take a seller whose `receive` writes one new storage slot, such as a contract wallet that keeps a record of who paid it.

- **Gas limit up to 16,777,216.** The transaction has no reservoir, so the wallet pays for the slot out of its own 102,300 gas. The slot alone needs 110,020, so the wallet runs out of gas before it is done. The ETH call fails, and the seller is paid in WETH, whatever gas limit the buyer sets in this range.

- **Gas limit well above 16,777,216.** The gas above the cap fills the reservoir, and the reservoir reaches the wallet in full, past the 100,000 cap. It pays the 97,920 for the new state, so the slot needs only 12,100 of the wallet's 102,300, and the seller is paid in ETH. The fill's own new state, such as the nonce it raises, is paid from the reservoir first, so the gas limit has to leave enough in it for the seller's slot as well.

	Current rules	Glamsterdam, no reservoir	Glamsterdam, with reservoir
Gas the wallet can use	102,300	102,300	102,300, plus the reservoir
Gas its new slot takes from that	22,100	110,020	12,100
The seller is paid in	ETH	WETH	ETH

So the same listing, filled with the same calldata, pays the seller in WETH or in ETH depending only on the gas limit the buyer picks. No value is lost, but a seller contract that cannot move ERC-20 tokens would be left holding WETH it cannot use.

What we tested

We tested on our own contracts with Foundry under `--hardfork amsterdam`, which applies EIP-8037 and EIP-8038. The sellers were contract wallets that sign through ERC-1271, and every fill used the same listing and the same calldata.

Seller's `receive`	Current rules	Glamsterdam, no reservoir	Glamsterdam, with reservoir
Writes one new storage slot	ETH	WETH	ETH
Only emits an event	ETH	ETH	ETH

The first two columns can be rerun from this repository. At commit 3aa9c61, `forge test` passes all 177 unit tests, and `forge test --hardfork amsterdam` fails 6 of them. One of the six is `test_normalContractWalletSeller_paidInEth` in `test/unit/VerifySellerPayoutGasCap.t.sol`. Its seller does some bookkeeping and writes one new storage slot when it is paid, and under the Glamsterdam rules it is paid in WETH. The last column follows from the reservoir rule of EIP-8037. A plain `forge test` run has no reservoir, so we checked it with a replay on a fork of a local deployment.

Three fixes we compared

We tested the escrow fallback and the revert against the code before the fix, on fills of 1,000,000 gas under the Glamsterdam rules.

Seller's `receive`	Before the fix	Escrow fallback	Revert, chosen
Writes one new storage slot	WETH	ETH to claim later	ETH
Only emits an event	ETH	ETH	ETH
Rejects ETH	WETH	ETH to claim later	the fill reverts

- **Raise the gas budget.** We rejected it, because it moves the line instead of removing it. A `receive` that writes more new slots still fails, the reservoir still flips the outcome, a hostile seller can burn more of the buyer's gas, and the next repricing can break it again.
- **Credit the seller in the escrow.** When the ETH payment fails, the marketplace would credit the seller in `CofferRedemptionEscrow`, which the core contracts already use for validator redemptions. Its `deposit` requires the ETH for every credit, and `claim` lets a contract seller send the ETH elsewhere, so the marketplace could share it without changes. It works. We did not choose it, because the buyer's gas limit would still decide between paid now and paid later, the fixed budget would stay, and sellers would need a separate claim step.
- **Revert the fill.** We chose it. A failed payment reverts the whole fill, so the payout has no branch on the result and no fixed gas budget to recheck at the next fork.

The fix

```
// A seller that does not accept the ETH fails the whole fill, so the nonce bump reverts with it
// and the listing stays live.
// slither-disable-next-line arbitrary-send-eth,calls-loop
require(_safeTransferETH(_seller, _price), SellerPaymentFailed());
```

- The payout forwards all remaining gas, less the 1/64 the EVM keeps back. A seller that rejects the ETH or runs out of gas fails the fill with `SellerPaymentFailed()`. Nothing moves, and the listing stays live.
- `SELLER_PAYOUT_GAS_LIMIT`, the WETH fallback, the gas parameter of `_safeTransferETH` and the unused `deposit()` in `IWETH` are removed.
- `_safeTransferETH` still copies no return data, so a seller that returns a large payload adds no copying cost to the fill.
- Offers do not change and still settle in WETH.
- The core contracts already pay this way. Every ETH payment there goes through `Address.sendValue`, which forwards all gas and reverts when the payment fails.

What we accept. A contract seller whose wallet rejects ETH can no longer sell through a listing. It can still sell by accepting an offer, which pays in WETH. Its listings stay live while every fill reverts, and they become fillable at their signed price once the wallet accepts ETH again, so such a seller should cancel them on-chain. A seller's `receive` can also burn the buyer's gas, which a contract wallet's signature check could already do, and the buyer's submitted gas limit is still the most a fill can cost.

Tests

On the fixed code, with Foundry 1.8.6.

- `forge test` passes all 173 unit tests under the current rules.
- `forge test --hardfork amsterdam` passes 172 of the 173. The one failure is explained below.
- The invariant suite passes under both rule sets, with no reverts in any handler.
- Slither reports no results.

`test/unit/VerifySellerPayout.t.sol` pins the new payout with these tests.

- `test_burningSeller_revertsWholeAndListingStaysLive`
- `test_normalContractWalletSeller_paidInEth`
- `test_expensiveContractWalletSeller_paidInEth`
- `test_freshEoaSeller_paidInEth`
- `testFuzz_sellerPayout_paidInEthOrNothing`, which pins that the buyer's gas limit decides only whether a fill settles, never what the seller is paid in

The one failure under the Glamsterdam rules is `test_poc6_massCancel1000Entries`. It checks that cancelling 1,000 listings in one call stays under 30 million gas. Under Glamsterdam the first cancel of each bond writes a new storage slot at about 110,000 gas, so the call needs about 113 million. That is a cost change and not a fault. `cancelAllListings` and `cancelAllOffers` revoke all of a maker's listings or offers with one write.

Core contracts

Glamsterdam does not change how the core contracts behave (`Coffer`, `CofferFactory`, `CofferBondNft`, `CofferRedemptionEscrow` and `FeeCurve`). None of them branches on a failed call, sets a fixed gas budget or reads `gasleft()`. Every external call forwards all gas and reverts the transaction when it fails. All 449 of their unit tests pass under both rule sets. Their gas costs rise wherever they create new state, so callers should rely on gas estimates rather than fixed limits.

References

- ChainSecurity, "Glamsterdam Hard Fork: When Spent Gas Comes Back", 13 October 2026
- EIP-8037, State Creation Gas Cost Increase
- EIP-8038, State-Access Gas Cost Update
- EIP-7825, Transaction Gas Limit Cap
- EIP-7773, the EIPs scheduled for Glamsterdam
- The Zealynx audit report, Coffer-Zealynx-Audit-Report.pdf, next to this document